

11. 排序

【本节目标】

1. 掌握七大基于比较的排序算法基本原理及实现
2. 掌握排序算法的性能分析
3. 掌握 Java 中的常用排序方法

1. 排序的概念及引用

1.1 排序的概念

排序：所谓排序，就是使一串记录，按照其中的某个或某些关键字的大小，递增或递减的排列起来的操作。

稳定性：假定在待排序的记录序列中，存在多个具有相同的关键字的记录，若经过排序，这些记录的相对次序保持不变，即在原序列中， $r[i]=r[j]$ ，且 $r[i]$ 在 $r[j]$ 之前，而在排序后的序列中， $r[i]$ 仍在 $r[j]$ 之前，则称这种排序算法是稳定的；否则称为不稳定的。

9 5_a 2 7 3 6 4 5_b 8 0



排序后

0 2 3 4 5_a 5_b 6 7 8 9 稳定

0 2 3 4 5_b 5_a 6 7 8 9 不稳定

内部排序：数据元素全部放在内存中的排序。

外部排序：数据元素太多不能同时放在内存中，根据排序过程的要求不断的在内外存之间移动数据的排序。

1.2 排序运用



1.3 常见的排序算法



2. 常见排序算法的实现

2.1 插入排序

2.1.1 基本思想：

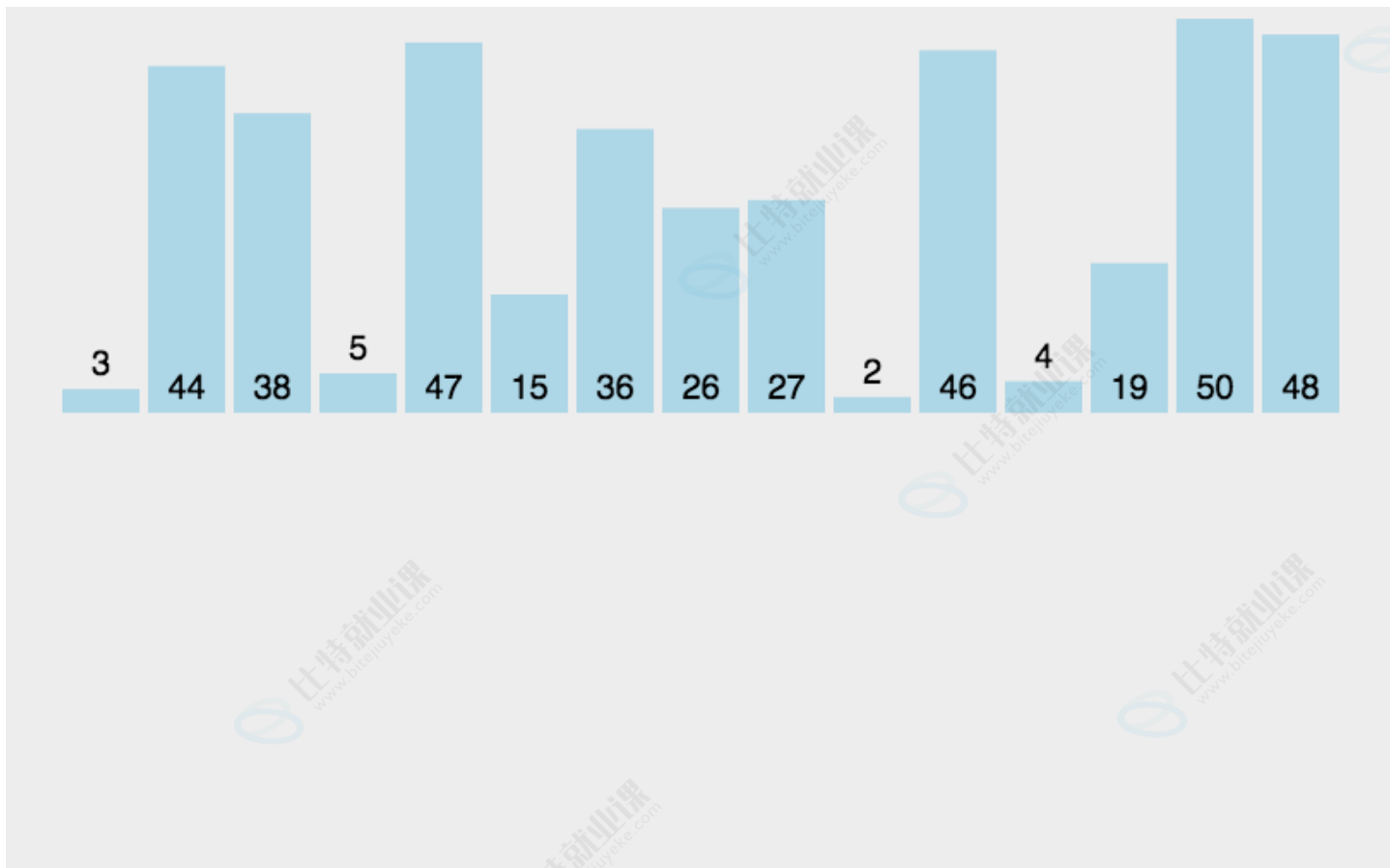
直接插入排序是一种简单的插入排序法，其基本思想是：

把待排序的记录按其关键码值的大小逐个插入到一个已经排好序的有序序列中，直到所有的记录插入完为止，得到一个新的有序序列。实际中我们玩扑克牌时，就用了插入排序的思想。



2.1.2 直接插入排序

当插入第 i ($i \geq 1$) 个元素时，前面的 $array[0], array[1], \dots, array[i-1]$ 已经排好序，此时用 $array[i]$ 的排序码与 $array[i-1], array[i-2], \dots$ 的排序码顺序进行比较，找到插入位置即将 $array[i]$ 插入，原来位置上的元素顺序后移

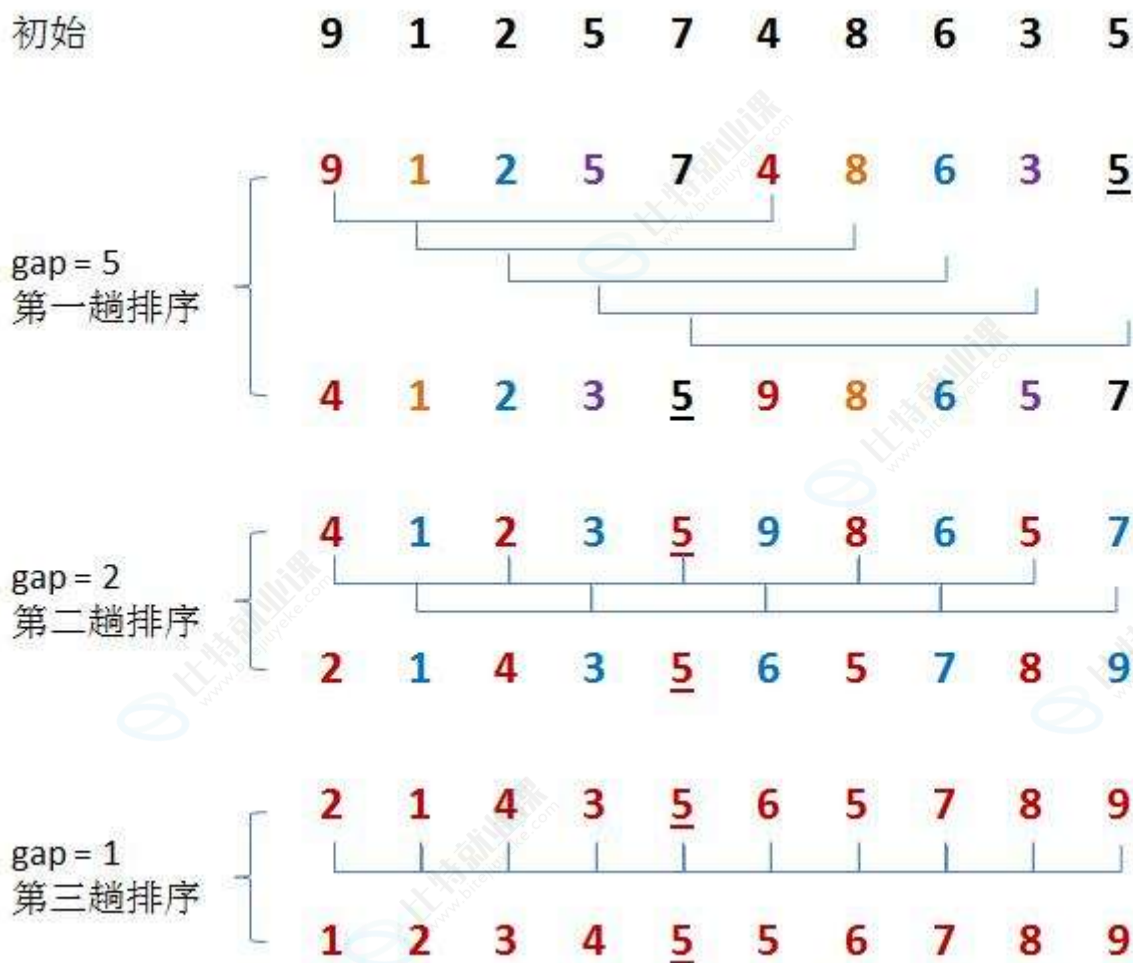


直接插入排序的特性总结：

1. 元素集合越接近有序，直接插入排序算法的时间效率越高
2. 时间复杂度： $O(N^2)$
3. 空间复杂度： $O(1)$ ，它是一种稳定的排序算法
4. 稳定性：稳定

2.1.3 希尔排序(缩小增量排序)

希尔排序法又称缩小增量法。希尔排序法的基本思想是：先选定一个整数，把待排序文件中所有记录分成多个组，所有距离为的记录分在同一组内，并对每一组内的记录进行排序。然后，取，重复上述分组和排序的工作。当到达=1时，所有记录在统一组内排好序。



希尔排序的特性总结：

1. 希尔排序是对直接插入排序的优化。
2. 当gap > 1时都是预排序，目的是让数组更接近于有序。当gap == 1时，数组已经接近有序的了，这样就会很快。这样整体而言，可以达到优化的效果。我们实现后可以进行性能测试的对比。

2.1.3.1 希尔排序的时间复杂度计算

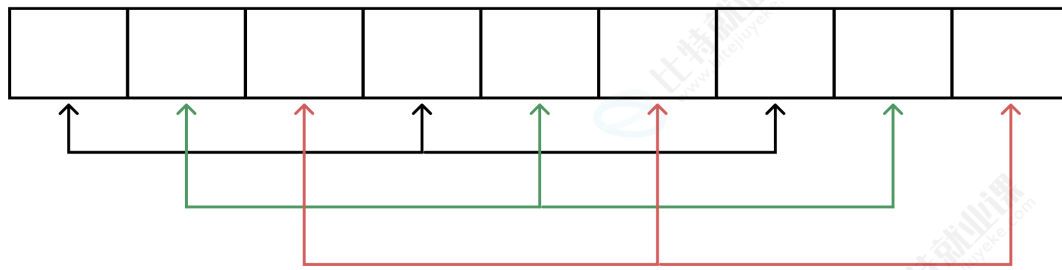
希尔排序的时间复杂度估算：

外层循环：

外层循环的时间复杂度可以直接给出为： $O(\log_2 n)$ 或者 $O(\log_3 n)$ ，即 $O(\log n)$

内层循环：

假设 $n = 9$ ，若 gap 为3，则一共 gap 组，每组 n/gap 个数据



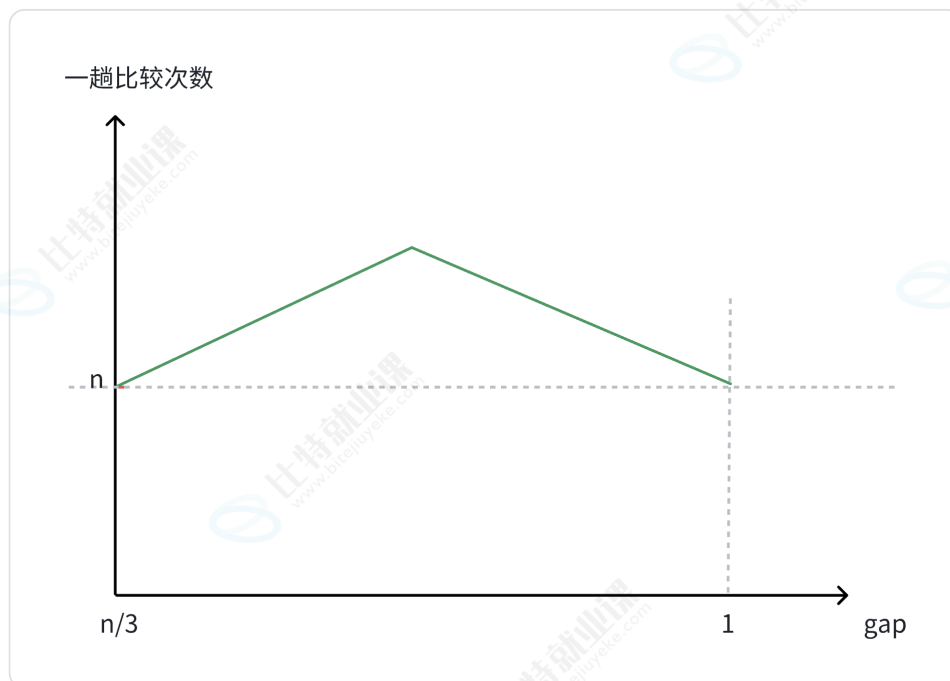
假设一共有 n 个数据，合计 gap 组，则每组为 n/gap 个；在每组中，插入移动的次数最坏的情况下为 $1 + 2 + 3 + \dots + (\frac{n}{gap} - 1)$ ，一共是 gap 组，因此：

总计最坏情况下移动总数为： $gap * [1 + 2 + 3 + \dots + (\frac{n}{gap} - 1)]$

gap 取值有（以除3为例）： $n/3 \ n/9 \ n/27 \ \dots \ 2 \ 1$

- 当 gap 为 $n/3$ 时，移动总数为： $\frac{n}{3} * (1 + 2) = n$
- 当 gap 为 $n/9$ 时，移动总数为： $\frac{n}{9} * (1 + 2 + 3 + \dots + 8) = \frac{n}{9} * \frac{8(1+8)}{2} = 4n$
- 最后一趟， $gap=1$ 即直接插入排序，内层循环排序消耗为 n

通过以上的分析，可以画出这样的曲线图：



因此，希尔排序在最初和最后的排序的次数都为 n ，即前一阶段排序次数是逐渐上升的状态，当到达某一顶点时，排序次数逐渐下降至 n ，而该顶点的计算暂时无法给出具体的计算过程

希尔排序时间复杂度不好计算，因为 gap 的取值很多，导致很难去计算，因此很多书中给出的希尔排序的时间复杂度都不固定。《数据结构(C语言版)》--- 严蔚敏书中给出的时间复杂度为：

gap 的取法有多种。最初 Shell 提出取 $gap = \lfloor n/2 \rfloor$, $gap = \lfloor gap/2 \rfloor$, 直到 $gap = 1$, 后来 Knuth 提出取 $gap = \lfloor gap/3 \rfloor + 1$ 。还有人提出都取奇数为好, 也有人提出各 gap 互质为好。无论哪一种主张都没有得到证明。

对希尔排序的时间复杂度的分析很困难, 在特定情况下可以准确地估算关键码的比较次数和对象移动次数, 但想要弄清关键码比较次数和对象移动次数与增量选择之间的依赖关系, 并给出完整的数学分析, 还没有人能够做到。在 Knuth 所著的《计算机程序设计技巧》第 3 卷中, 利用大量的实验统计资料得出, 当 n 很大时, 关键码平均比较次数和对象平均移动次数大约在 $n^{1.25}$ 到 $1.6n^{1.25}$ 范围内, 这是在利用直接插入排序作为子序列排序方法的情况下得到的。

希尔排序的分析是一个复杂的问题, 因为它的时间是所取“增量”序列的函数, 这涉及一些数学上尚未解决的难题。因此, 到目前为止尚未有人求得一种最好的增量序列, 但大量的研究已得出一些局部的结论。如有人指出, 当增量序列为 $dlt_a[k] = 2^{t-k+1} - 1$ 时, 希尔排序的时间复杂度为 $O(n^{3/2})$, 其中 t 为排序趟数, $1 \leq k \leq t \leq \lfloor \log_2(n+1) \rfloor$ 。还有人在大量的实验基础上推出: 当 n 在某个特定范围内, 希尔排序所需的比较和移动次数约为 $n^{1.3}$, 当 $n \rightarrow \infty$ 时, 可减少到 $n(\log_2 n)^{2[2]}$ 。增量序列可以有各种取法^①, 但需注意: 应使增量序列中的值没有除 1 之外的公因子, 并且最后一个增量值必须等于 1。

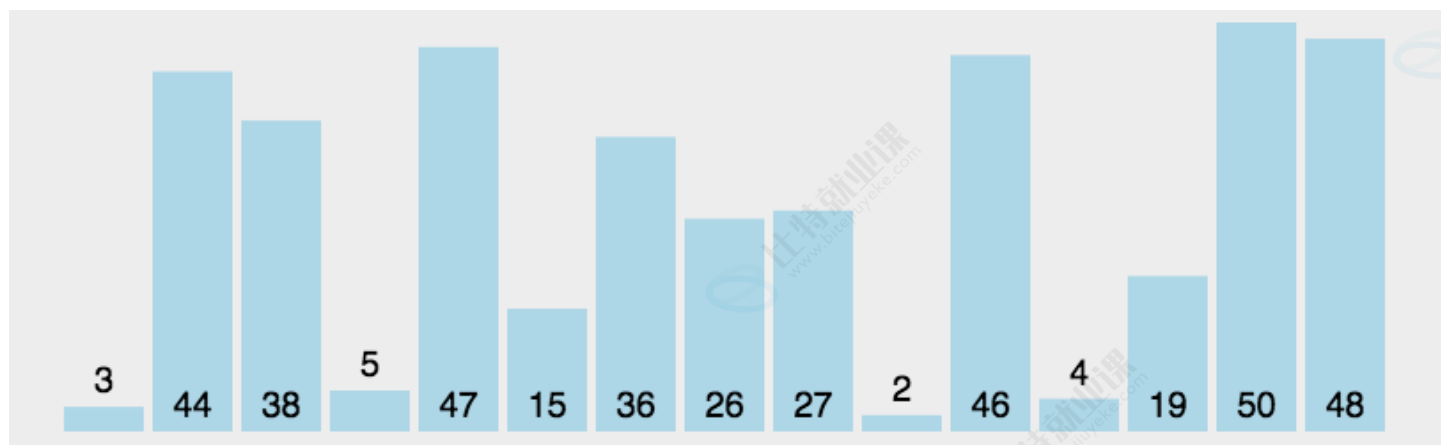
2.2 选择排序

2.2.1 基本思想:

每一次从待排序的数据元素中选出最小 (或最大) 的一个元素, 存放在序列的起始位置, 直到全部待排序的数据元素排完。

2.2.2 直接选择排序

- 在元素集合 $array[i] \dots array[n-1]$ 中选择关键码最大 (小) 的数据元素
- 若它不是这组元素中的最后一个 (第一个) 元素, 则将它与这组元素中的最后一个 (第一个) 元素交换
- 在剩余的 $array[i] \dots array[n-2]$ ($array[i+1] \dots array[n-1]$) 集合中, 重复上述步骤, 直到集合剩余 1 个元素

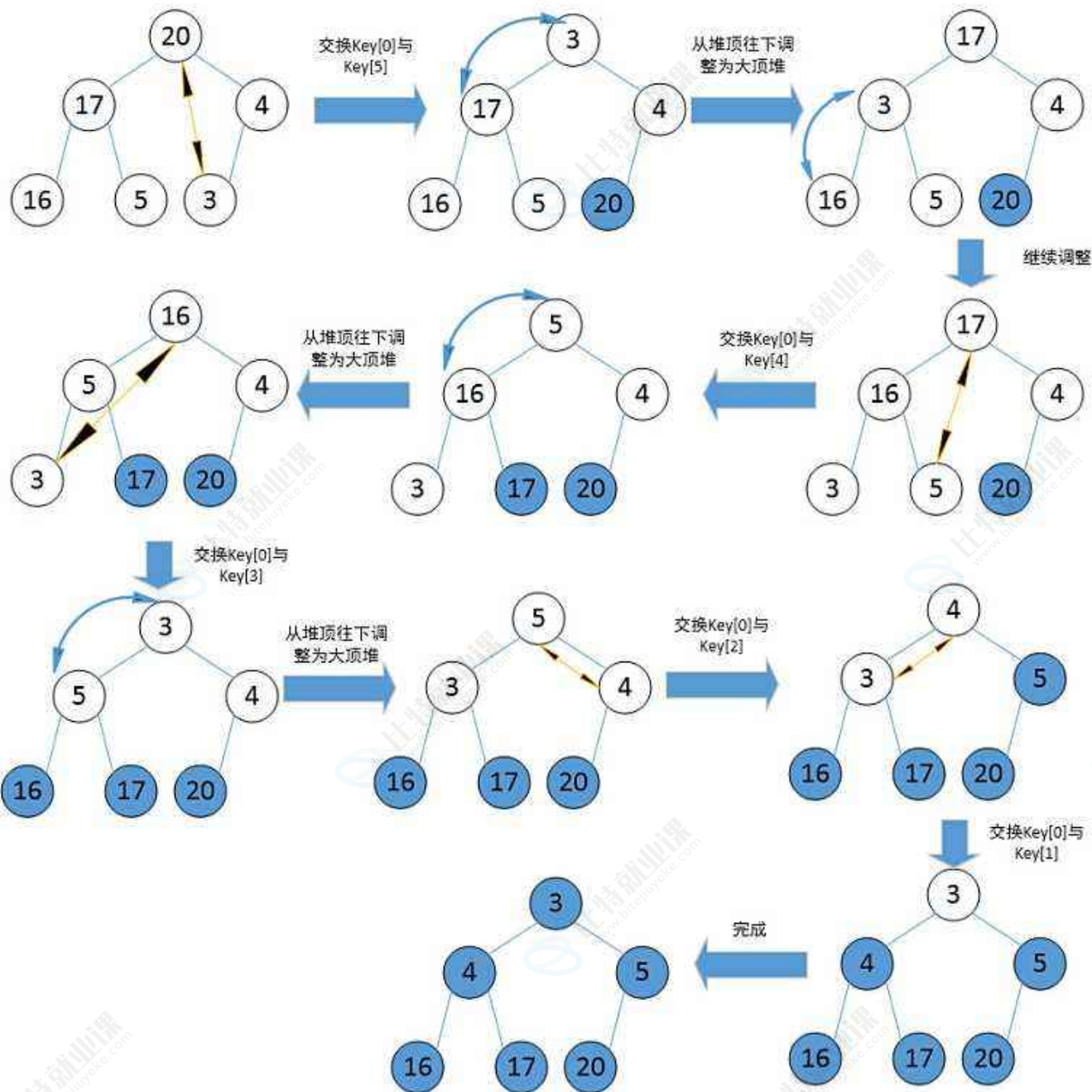


【直接选择排序的特性总结】

1. 直接选择排序思考非常好理解，但是效率不是很好。实际中很少使用
2. 时间复杂度： $O(N^2)$
3. 空间复杂度： $O(1)$
4. 稳定性：不稳定

2.2.3 堆排序

堆排序(Heapsort)是指利用堆积树（堆）这种数据结构所设计的一种排序算法，它是选择排序的一种。它是通过堆来进行选择数据。**需要注意的是排升序要建大堆，排降序建小堆。**



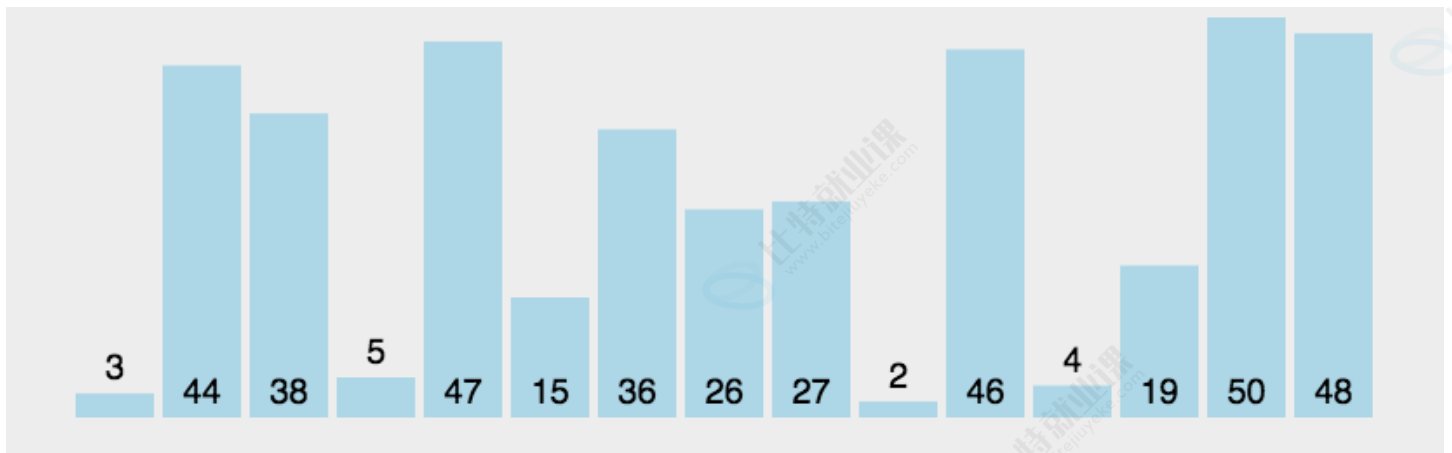
【堆排序的特性总结】

1. 堆排序使用堆来选数，效率就高了很多。
2. 时间复杂度： $O(N \cdot \log N)$
3. 空间复杂度： $O(1)$
4. 稳定性：不稳定

2.3 交换排序

基本思想：所谓交换，就是根据序列中两个记录键值的比较结果来对换这两个记录在序列中的位置，交换排序的特点是：将键值较大的记录向序列的尾部移动，键值较小的记录向序列的前部移动。

2.3.1 冒泡排序



【冒泡排序的特性总结】

1. 冒泡排序是一种非常容易理解的排序
2. 时间复杂度： $O(N^2)$
3. 空间复杂度： $O(1)$
4. 稳定性：稳定

2.3.2 快速排序

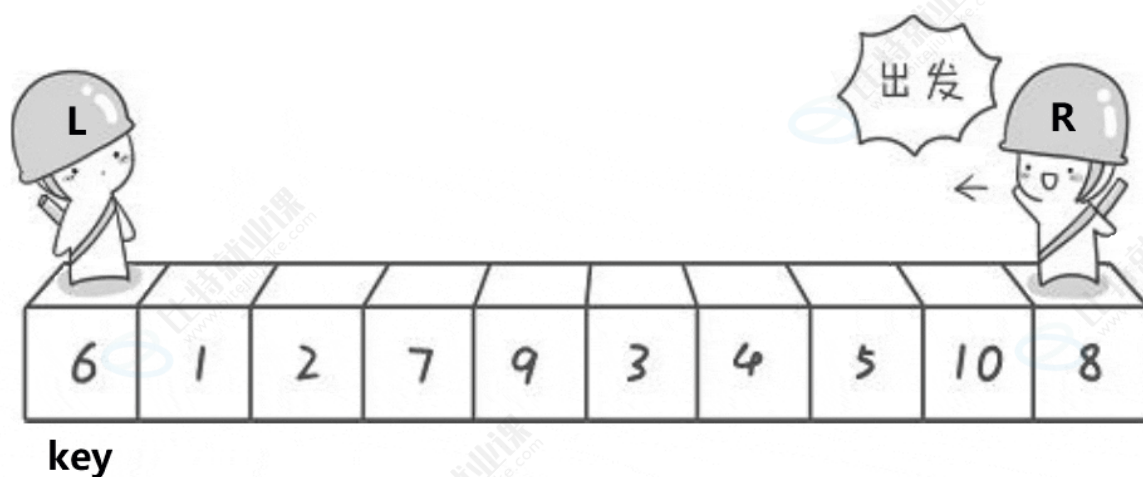
快速排序是Hoare于1962年提出的一种二叉树结构的交换排序方法，其基本思想为：**任取待排序元素序列中的某元素作为基准值，按照该排序码将待排序集合分割成两子序列，左子序列中所有元素均小于基准值，右子序列中所有元素均大于基准值，然后最左右子序列重复该过程，直到所有元素都排列在相应位置上为止。**

```
1 // 假设按照升序对array数组中[left, right)区间中的元素进行排序
2 void QuickSort(int[] array, int left, int right) {
3     if(right - left <= 1)
4         return;
5
6     // 按照基准值对array数组的 [left, right)区间中的元素进行划分
7     int div = partion(array, left, right);
8
9     // 划分成功后以div为边界形成了左右两部分 [left, div) 和 [div+1, right)
10    // 递归排[left, div)
11    QuickSort(array, left, div);
12
13    // 递归排[div+1, right)
14    QuickSort(array, div+1, right);
15 }
```

上述为快速排序递归实现的主框架，发现与二叉树前序遍历规则非常像，同学们在写递归框架时可想想二叉树前序遍历规则即可快速写出来，后序只需分析如何按照基准值来对区间中数据进行划分的方式即可。

将区间按照基准值划分为左右两半部分的常见方式有：

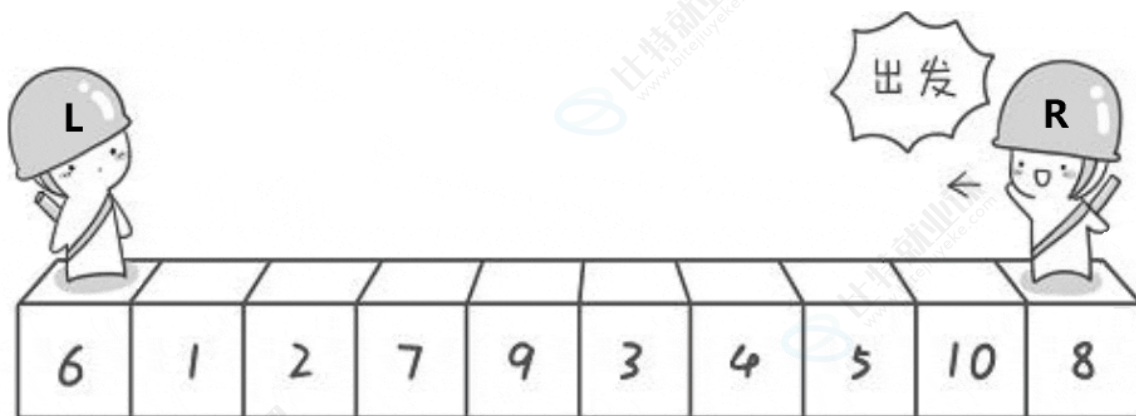
1. Hoare版



```
1 private static int partition(int[] array, int left, int right) {
2     int i = left;
3     int j = right;
4     int pivot = array[left];
5     while (i < j) {
6         while (i < j && array[j] >= pivot) {
7             j--;
8         }
9
10        while (i < j && array[i] <= pivot) {
11            i++;
12        }
13
14        swap(array, i, j);
15    }
16    swap(array, i, left);
17    return i;
18 }
```

2. 挖坑法

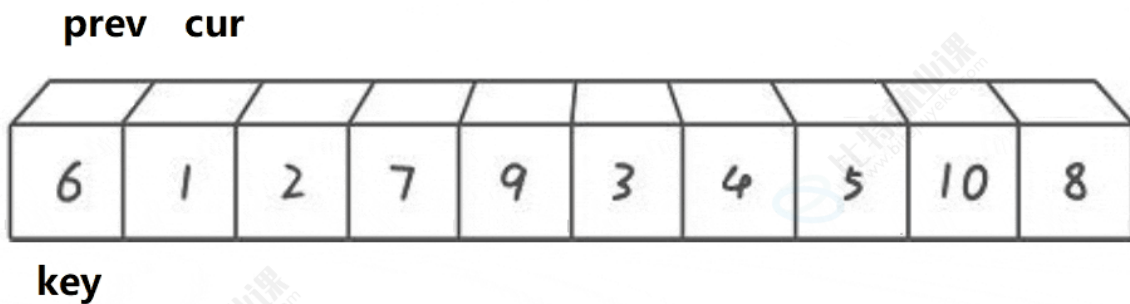
先将第一个数据存放在临时变量 key 中，形成一个坑位
key =



```
1 private static int partition(int[] array, int left, int right) {  
2     int i = left;  
3     int j = right;  
4     int pivot = array[left];  
5     while (i < j) {  
6         while (i < j && array[j] >= pivot) {  
7             j--;  
8         }  
9  
10        array[i] = array[j];  
11  
12        while (i < j && array[i] <= pivot) {  
13            i++;  
14        }  
15  
16        array[j] = array[i];  
17    }  
18    array[i] = pivot;  
19    return i;  
20 }
```

3. 前后指针

初始时, *prev*指针指向序列开头,
*cur*指针指向*prev*指针的后一个位置



```
1 private static int partition(int[] array, int left, int right) {
2     int prev = left ;
3     int cur = left+1;
4     while (cur <= right) {
5         if(array[cur] < array[left] && array[++prev] != array[cur]) {
6             swap(array,cur,prev);
7         }
8         cur++;
9     }
10    swap(array,prev,left);
11    return prev;
12 }
```

2.3.3 快速排序优化

1. 三数取中法选key
2. 递归到小的子区间时, 可以考虑使用插入排序

2.3.4 快速排序非递归

```
1 void quickSortNonR(int[] a, int left, int right) {
2     Stack<Integer> st = new Stack<>();
3     st.push(left);
4     st.push(right);
5     while (!st.empty()) {
6         right = st.pop();
7         left = st.pop();
8         if(right - left <= 1)
```

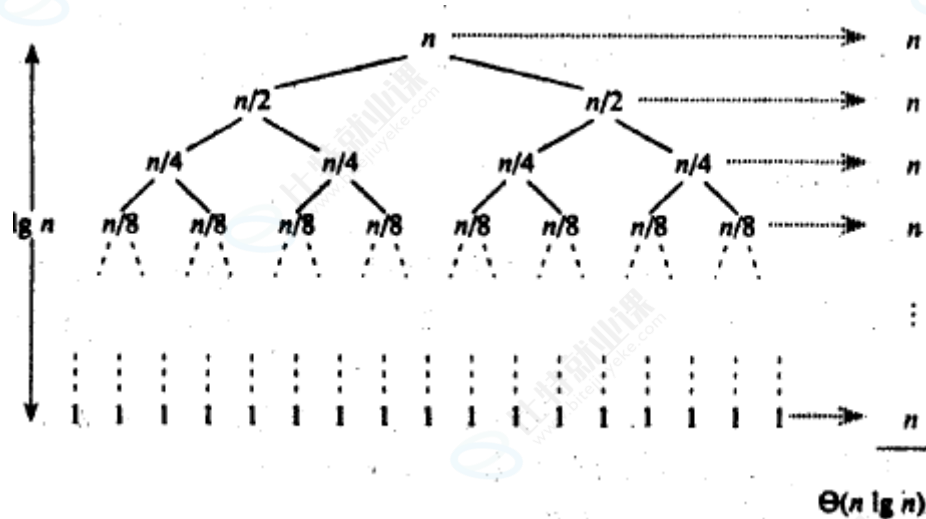
```

9         continue;
10    int div = PartSort1(a, left, right);
11    // 以基准值为分割点, 形成左右两部分: [left, div) 和 [div+1, right)
12    st.push(div+1);
13    st.push(right);
14
15    st.push(left);
16    st.push(div);
17 }
18 }

```

2.3.5 快速排序总结

1. 快速排序整体的综合性能和使用场景都是比较好的, 所以才敢叫**快速排序**
2. 时间复杂度: $O(N \cdot \log N)$

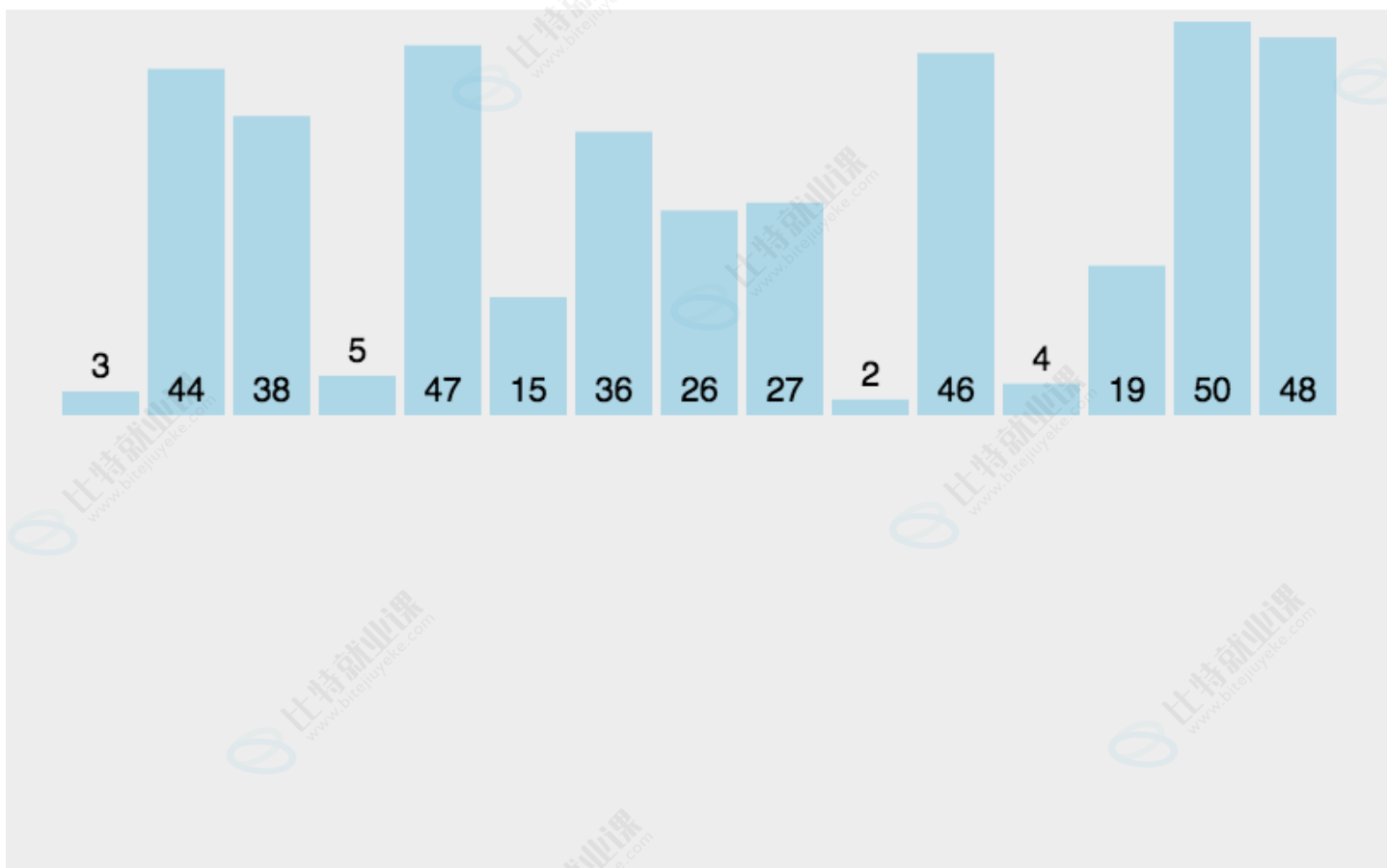
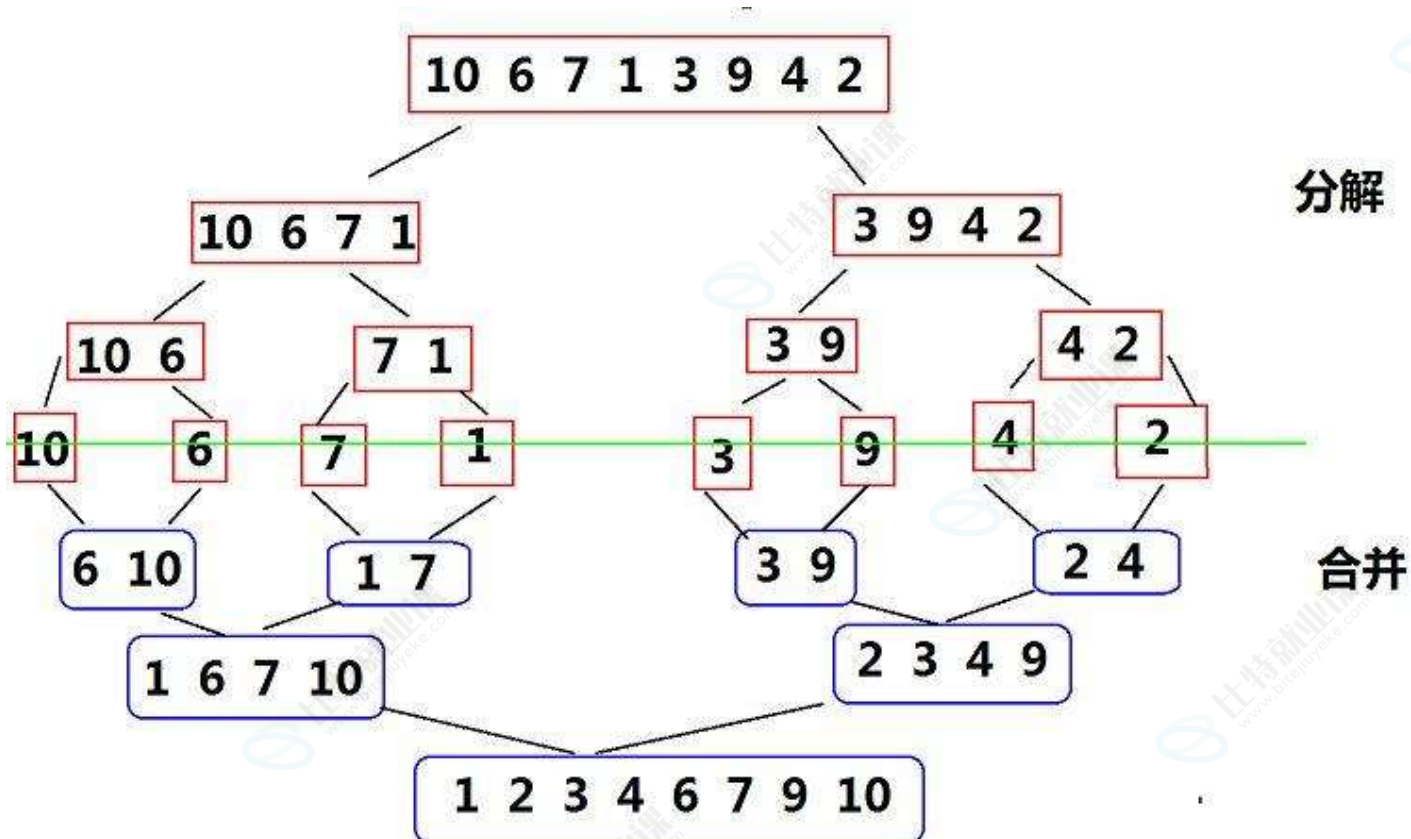


3. 空间复杂度: $O(\log N)$
4. 稳定性: 不稳定

2.4 归并排序

2.4.1 基本思想

归并排序 (MERGE-SORT) 是建立在归并操作上的一种有效的排序算法, 该算法是采用分治法 (Divide and Conquer) 的一个非常典型的应用。将已有序的子序列合并, 得到完全有序的序列; 即先使每个子序列有序, 再使子序列段间有序。若将两个有序表合并成一个有序表, 称为二路归并。归并排序核心步骤:



2.4.2 归并排序总结

1. 归并的缺点在于需要 $O(N)$ 的空间复杂度，归并排序的思考更多的是解决在磁盘中的外排序问题。
2. 时间复杂度： $O(N \cdot \log N)$
3. 空间复杂度： $O(N)$

4. 稳定性：稳定

2.4.3 海量数据的排序问题

外部排序：排序过程需要在磁盘等外部存储进行的排序

前提：内存只有 1G，需要排序的数据有 100G

因为内存中因为无法把所有数据全部放下，所以需要外部排序，而归并排序是最常用的外部排序

1. 先把文件切分成 200 份，每个 512 M
2. 分别对 512 M 排序，因为内存已经可以放的下，所以任意排序方式都可以
3. 进行 2路归并，同时对 200 份有序文件做归并过程，最终结果就有序了

3. 排序算法复杂度及稳定性分析



排序方法	最好	最坏	空间复杂度	稳定性
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
插入排序	$O(n)$	$O(n^2)$	$O(1)$	稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
希尔排序	$O(n)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n \cdot \log(n))$	$O(n \cdot \log(n))$	$O(1)$	不稳定
快速排序	$O(n \cdot \log(n))$	$O(n^2)$	$O(\log(n)) \sim O(n)$	不稳定
归并排序	$O(n \cdot \log(n))$	$O(n \cdot \log(n))$	$O(n)$	稳定

4. 其他非基于比较排序（了解）

1. 计数排序

思想：计数排序又称为鸽巢原理，是对哈希直接定址法的变形应用。操作步骤：

a. 统计相同元素出现次数

b. 根据统计的结果将序列回收到原来的序列中

【计数排序的特性总结】

a. 计数排序在数据范围集中时，效率很高，但是适用范围及场景有限。

b. 时间复杂度： $O(\text{MAX}(N, \text{范围}))$

c. 空间复杂度： $O(\text{范围})$

d. 稳定性：稳定

2. 基数排序

3. 桶排序

5. 选择题

1. 快速排序算法是基于()的一个排序算法。

A: 分治法 B: 贪心法 C: 递归法 D: 动态规划法

2. 对记录 (54, 38, 96, 23, 15, 72, 60, 45, 83) 进行从小到大的直接插入排序时，当把第8个记录45插入到有序表时，为找到插入位置需比较()次？（采用从后往前比较）

A: 3 B: 4 C: 5 D: 6

3. 以下排序方式中占用 $O(n)$ 辅助存储空间的是()

A: 简单排序 B: 快速排序 C: 堆排序 D: 归并排序

4. 下列排序算法中稳定且时间复杂度为 $O(n^2)$ 的是()

A: 快速排序 B: 冒泡排序 C: 直接选择排序 D: 归并排序

5. 关于排序，下面说法不正确的是()

A: 快排时间复杂度为 $O(N \cdot \log N)$ ，空间复杂度为 $O(\log N)$

B: 归并排序是一种稳定的排序，堆排序和快排均不稳定

C: 序列基本有序时，快排退化成 "冒泡排序"，直接插入排序最快

D: 归并排序空间复杂度为 $O(N)$ ，堆排序空间复杂度的为 $O(\log N)$

6. 设一组初始记录关键字序列为 (65, 56, 72, 99, 86, 25, 34, 66)，则以第一个关键字65为基准而得到的一趟快速排序结果是 ()

A: 34, 56, 25, 65, 86, 99, 72, 66

B: 25, 34, 56, 65, 99, 86, 72, 66

C: 34, 56, 25, 65, 66, 99, 86, 72

D: 34, 56, 25, 65, 99, 86, 72, 66

23

24 【参考答案】

25 1.A 2.C 3.D 4.B 5.D 6.A

完